

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬ-
НОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 1

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса

Обучающийся Доценников Никита Андреевич

Факультет Прикладной информатики

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Доценников Н.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Содержание

Персональные параметры	3
Этап 1. Верификация окружения и архитектуры	4
Задание 1.1	4
Задание 1.2	4
Задание 1.3	5
Этап 2. Первые шаги: «игрушечные» образы	7
Задание 2.1	7
Задание 2.2	7
Задание 2.3	8
Этап 3. Жизненный цикл контейнера	11
Задание 3.1	11
Задание 3.2	12
Этап 4. Диагностика реального сервиса	15
Задание 4.1	15
Задание 4.2	16
Задание 4.3	17
Вывод	18

Персональные параметры

Параметр	Значение
Номер ИСУ	<i>465797</i>
Персональный порт	<i>5527</i>
Персональный префикс	<i>doschennikov</i>

Таблица 1. Персональные параметры

Этап 1. Верификация окружения и архитектуры

Задание 1.1

Я проверил установку Docker командой:

```
docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
```

```
[doschennikov@archlinux ~]$ docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
```

```
Client: 29.8.1, Server: 29.8.1
```

```
[doschennikov@archlinux ~]$ _
```

Рисунок 1. Проверка установки Docker

- Версия клиента: 29.8.1
- Версия сервера: 29.8.1

Я узнал тип ОС и архитектуры при помощи:

```
docker info --format "{{.OSType}}/{{.Architecture}}"
```

Вывод:

```
linux/x86_64
```

```
[doschennikov@archlinux ~]$ docker info --format "{{.OSType}}/{{.Architecture}}"
```

```
linux/x86_64
```

```
[doschennikov@archlinux ~]$
```

Рисунок 2. Проверка типа ОС и архитектуры

Задание 1.2

Я исследовал состояние локального хранилища командой:

```
docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
```

```
[doschennikov@archlinux ~]$ docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
```

TYPE	TOTAL	ACTIVE	SIZE
Images	0	0	0B
Containers	0	0	0B
Local Volumes	0	0	0B
Build Cache	0	0	0B

```
[doschennikov@archlinux ~]$
```

Рисунок 3. Локальное хранилище

- До начала работы хранится 0 образов.
- Сейчас активно 0 запущенных контейнеров.

Задание 1.3

Затем я попытался вывести информацию о текущей сессии командой из условия работы:

```
echo "Student: $(whoami)@$(hostname)" && \  
echo "Date: $(date '+%Y-%m-%d %H:%M:%S')" && \  
docker version --format "Docker {{.Server.Version}} on {{.Server.Os}}/  
{{.Server.Architecture}}"
```

И получил следующую ошибку:

```
[doschennikov@archlinux ~]$ echo "Student: $(whoami)@$(hostname)" && echo "Date: $(d  
ate '+%Y-%m-%d %H:%M:%S')" && docker version --format "Docker {{.Server.Version}} on  
{{.Server.Os}}/{{.Server.Architecture}}"  
Student: doschennikov@archlinux  
Date: 2026-09-23 21:53:38  
Docker 29.8.1 on linux/  
template: version:1:54: executing "version" at <.Server.Architecture>: can't evaluat  
e field Architecture in type *system.serverVersion
```

Рисунок 4. Ошибка при попытке вывода информации

Дело в том, что у `.Server` отсутствует поле `.Architecture`. Я исправил команду следующим образом:

```
echo "Student: $(whoami)@$(hostname)" && \  
echo "Date: $(date '+%Y-%m-%d %H:%M:%S')" && \  
docker version --format "Docker {{.Server.Version}} on {{.Server.Os}}/  
{{(index .Server.Components 0).Details.Arch}}"
```

```
[doschennikov@archlinux ~]$ echo "Student: $(whoami)@$(hostname)" && echo "Date: $(d  
ate '+%Y-%m-%d %H:%M:%S')" && docker version --format "Docker {{.Server.Version}} on  
{{.Server.Os}}/{{(index .Server.Components 0).Details.Arch}}"  
Student: doschennikov@archlinux  
Date: 2026-09-23 21:51:26  
Docker 29.8.1 on linux/amd64
```

Рисунок 5. Удачная попытка

- Команда `docker system prune` без флагов удаляет остановленные контейнеры, неиспользуемые сети, а также только висячие («dangling») образы и соответствующий неиспользуемый кэш сборки. Локальные тома по умолчанию не удаляются, для их удаления необходимо дополнительно указать флаг `--volumes`. Перед удалением Docker запрашивает подтверждение пользователя. Таким образом, могут быть потеряны остановленные контейнеры, неиспользуемые сети, висячие образы и часть кэша сборки.
- `docker version` может завершиться ошибкой в основном по нескольким причинам:
 - Не установлены параметры окружения. Информацию об этой проблеме я нашел здесь.

- ▶ Не включен `docker.service` . Включить можно командой:

```
systemctl enable --now docker.service
```

```
[doschennikov@archlinux ~]$ docker version
Client:
Version:      29.8.1
API version:  1.56
Go version:   go1.27.1-X:nodwarf5
Git commit:   4a63305d74
Built:        Fri Sep 18 15:27:28 2026
OS/Arch:      linux/amd64
Context:      default
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker da
emon running?
[doschennikov@archlinux ~]$ echo $?
1
[doschennikov@archlinux ~]$
```

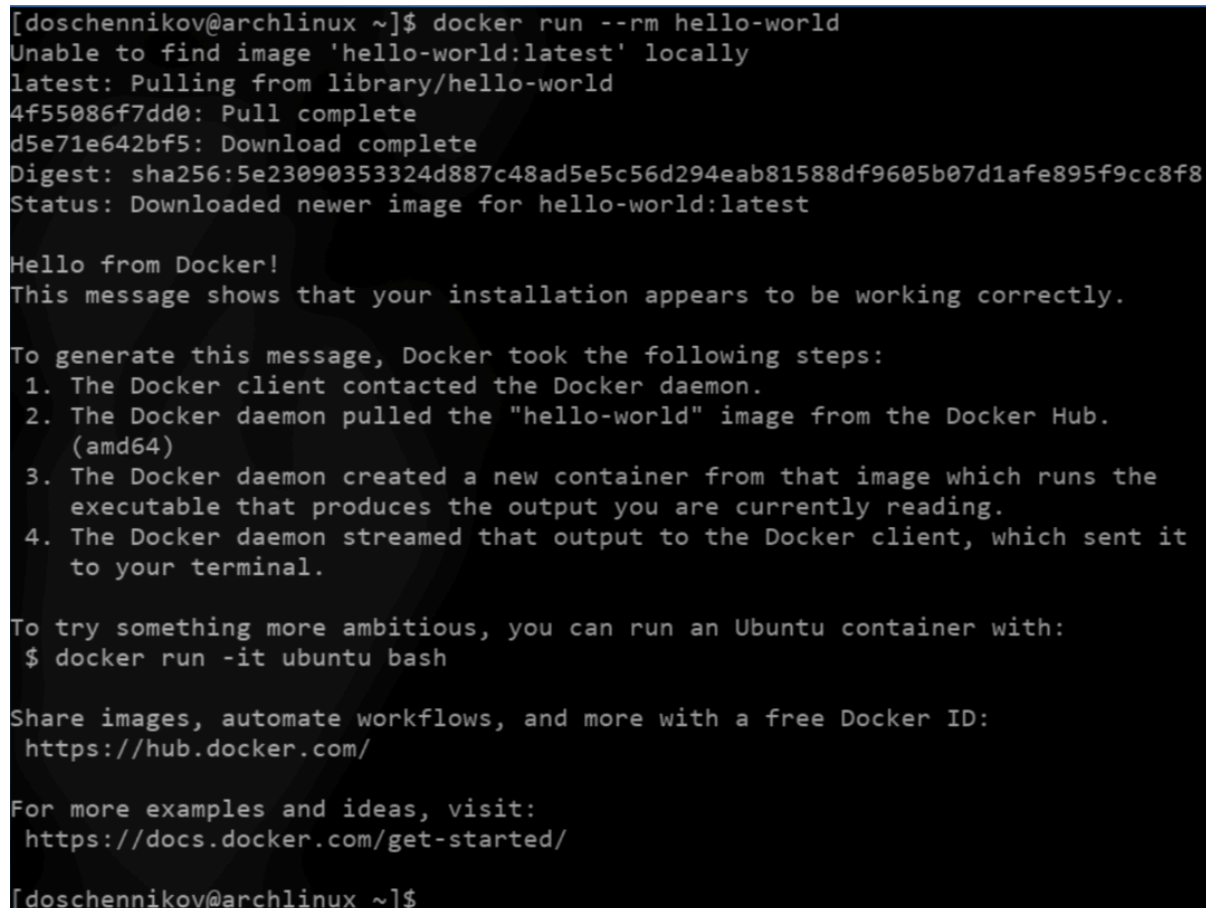
Рисунок 6. Ошибка при вызове `docker version` .

Этап 2. Первые шаги: «игрушечные» образы

Задание 2.1

Я запустил классический пример командой:

```
docker run --rm hello-world
```



```
[doschennikov@archlinux ~]$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[doschennikov@archlinux ~]$
```

Рисунок 7. Результат исполнения hello-world.

- Образ загружается из Docker Hub: `docker.io/library/hello-world:latest`.
- Внутри контейнера выполняется бинарный файл `/hello`. Он является главным процессом контейнера (PID 1), выводит сообщение `Hello from Docker!` и после этого завершается.
- Последовательность работы выглядит следующим образом: Docker проверяет наличие образа в локальном кэше. Если образ отсутствует, он загружается из Docker Hub. После этого Docker создаёт контейнер и запускает в нём `/hello`. Вывод процесса передаётся Docker-клиенту и отображается в терминале.

Задание 2.2

Я исследовал минимальный образ:

```
docker run --rm alpine:3.18 cat /etc/os-release
```

```
[doschennikov@archlinux ~]$ docker run --rm alpine:3.18 cat /etc/os-release
Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
44cf07d57ee4: Pull complete
431d356fe850: Download complete
54d130656ede: Download complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.18.12
PRETTY_NAME="Alpine Linux v3.18"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
[doschennikov@archlinux ~]$
```

Рисунок 8. Вывод `/etc/os-release` образа `alpine:3.18`

Затем я посмотрел размер образа:

```
docker images alpine:3.18 --format "{{.Size}}"
```

```
[doschennikov@archlinux ~]$ docker images alpine:3.18 --format "{{.Size}}"
12.5MB
[doschennikov@archlinux ~]$
```

Рисунок 9. Размер образа `alpine:3.18`

- Значение поля `Pretty_Name` : «Alpine Linux v3.18»
- Размер образа: 12.5 MB

Задание 2.3

Я сравнил поведение двух контейнеров:

```
docker run -d --name doschennikov-alpine-1 alpine:3.18 sleep 300
docker run -d --name doschennikov-alpine-2 alpine:3.18 sh -c "echo
'Hello from $(hostname)' && sleep 300"
```

И проверил их статус:

```
docker ps --filter "name=doschennikov-alpine"
```

```
[doschennikov@archlinux ~]$ docker run -d --name doschennikov-alpine-1 alpine:3.18 sleep 300
8316a5856f7a0457ceabc51d702a6cc06151b9f07a9f02dde05b2187aba982c1
[doschennikov@archlinux ~]$ docker run -d --name doschennikov-alpine-2 alpine:3.18 sh -c "echo 'Hello from $(hostname)' && sleep 300"
3a375ec9cab32c1fbe655ddca3777560e34ac92775d5d771d28eae93a1f08329
[doschennikov@archlinux ~]$
[doschennikov@archlinux ~]$ docker ps --filter "name=doschennikov-alpine"
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
PORTS         NAMES
3a375ec9cab3   alpine:3.18  "sh -c 'echo 'Hello ..."  59 seconds ago Up 58 seconds
doschennikov-alpine-2
8316a5856f7a   alpine:3.18  "sleep 300"             About a minute ago Up About a minute
doschennikov-alpine-1
[doschennikov@archlinux ~]$
```

Рисунок 10. Статус контейнеров

Второй контейнер ожидаемо выводит сообщение:

```
docker logs doschennikov-alpine-1
docker logs doschennikov-alpine-2
```

```
[doschennikov@archlinux ~]$ docker logs doschennikov-alpine-1
[doschennikov@archlinux ~]$ docker logs doschennikov-alpine-2
Hello from archlinux
[doschennikov@archlinux ~]$
```

Рисунок 11. Логи контейнеров

- Контейнер `hello-world` завершается сразу после запуска, потому что его главный процесс (PID 1) `/hello` выполняет свою задачу, выводит сообщение и завершается. Когда главный процесс контейнера завершается, контейнер также переходит в состояние `Exited`. Флаг `--rm` не влияет на продолжительность работы контейнера, а только указывает Docker автоматически удалить контейнер после его завершения. В случае `alpine:3.18` главным процессом является команда `sleep 300`, которая не завершается в течение 300 секунд, поэтому контейнер остаётся в состоянии `Up` примерно 5 минут.
- Если убрать флаг `--rm` из команды `docker run --rm hello-world`, контейнер всё равно завершится после завершения процесса `/hello`. Отличие заключается в том, что Docker не удалит его автоматически, а контейнер останется в состоянии `Exited` и будет отображаться командой `docker ps -a`. Удалить такие остановленные контейнеры можно командой `docker container prune` или удалить конкретный контейнер командой `docker rm`.

```
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[doschennikov@archlinux ~]$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	Info →	U	In Use
alpine:3.18	de0eb0b3f2a4	12.5MB	3.5MB	U		
hello-world:latest	5e2309035332	21.8kB	9.49kB	U		

```
[doschennikov@archlinux ~]$
```

Рисунок 12. Оставшиеся образы

```
[doschennikov@archlinux ~]$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
f0e31e4a81307bddff2f864c3397dba85d3c2031e2a12f6d3acc276bb49e37fe
7936a798a7d69f4cf92e9b45aae8fca02de26ab1defd933d62494583d1ca001b
3a375ec9cab32c1fbe655ddca3777560e34ac92775d5d771d28eae93a1f08329
8316a5856f7a0457ceabc51d702a6cc06151b9f07a9f02dde05b2187aba982c1

Total reclaimed space: 0B
[doschennikov@archlinux ~]$
```

Рисунок 13. Удаление оставленных контейнеров

Этап 3. Жизненный цикл контейнера

Задание 3.1

Я проследил все состояния через последовательные команды:

1. Создание без запуска

```
docker create --name doschennikov-nginx nginx:alpine
```

```
[doschennikov@archlinux ~]$ docker create --name doschennikov-nginx nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
6c53d0b2a666: Pull complete
e76228b47809: Pull complete
e2de96513ba9: Pull complete
e72112c14215: Pull complete
9a9a644fdd6a: Pull complete
745dfb2690dd: Pull complete
64c8194480fe: Pull complete
d9aae54b5831: Pull complete
0deea27e58d9: Download complete
d54d3939625e: Download complete
Digest: sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
Status: Downloaded newer image for nginx:alpine
dc1c3184d55204e06d67d5a7bcd45ae2150a90730743b080f86ace57bc663da3
[doschennikov@archlinux ~]$ _
```

Рисунок 14. Создание без запуска

2. Запуск

```
docker start doschennikov-nginx
```

```
[doschennikov@archlinux ~]$ docker start doschennikov-nginx
doschennikov-nginx
[doschennikov@archlinux ~]$ _
```

Рисунок 15. Запуск контейнера

3. Остановка «мягкая» (сигнал SIGTERM)

```
docker stop doschennikov-nginx
```

```
[doschennikov@archlinux ~]$ docker stop doschennikov-nginx
doschennikov-nginx
[doschennikov@archlinux ~]$
```

Рисунок 16. Остановка контейнера «мягкая»

4. Перезапуск

```
docker restart doschennikov-nginx
```

```
[doschennikov@archlinux ~]$ docker restart doschennikov-nginx
doschennikov-nginx
[doschennikov@archlinux ~]$ _
```

Рисунок 17. Перезапуск контейнера

5. Остановка «жесткая» (сигнал `SIGKILL`)

```
docker kill doschennikov-nginx
```

```
[doschennikov@archlinux ~]$ docker kill doschennikov-nginx
doschennikov-nginx
[doschennikov@archlinux ~]$ _
```

Рисунок 18. Остановка контейнера «жесткая»

6. Удаление

```
docker rm doschennikov-nginx
```

```
[doschennikov@archlinux ~]$ docker rm doschennikov-nginx
doschennikov-nginx
[doschennikov@archlinux ~]$ _
```

Рисунок 19. Удаление контейнера

Задание 3.2

Я зафиксировал переходы состояний командой:

```
docker ps -a --filter "name=doschennikov-nginx" --format "table
{{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
```

1. До создания контейнера:

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES      STATUS      CREATED AT
[doschennikov@archlinux ~]$
```

Рисунок 20. Состояние до создания контейнера

2. После создания контейнера:

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES      STATUS      CREATED AT
doschennikov-nginx  Created    2026-09-29 15:17:44 +0300 MSK
[doschennikov@archlinux ~]$ _
```

Рисунок 21. Состояние после создания контейнера

3. После запуска:

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS          CREATED AT
doschennikov-nginx Up 2 seconds    2026-09-29 15:17:44 +0300 MSK
[doschennikov@archlinux ~]$ _
```

Рисунок 22. Состояние после запуска контейнера

4. После «мягкой» остановки (SIGTERM):

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS          CREATED AT
doschennikov-nginx Exited (0) 1 second ago 2026-09-29 15:17:44 +0300 MSK
[doschennikov@archlinux ~]$ _
```

Рисунок 23. Состояние после остановки SIGTERM

5. После перезапуска:

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS          CREATED AT
doschennikov-nginx Up 1 second    2026-09-29 15:17:44 +0300 MSK
[doschennikov@archlinux ~]$ _
```

Рисунок 24. Состояние после перезапуска

6. После «жесткой» остановки (SIGKILL):

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS          CREATED AT
doschennikov-nginx Exited (137) 2 seconds ago 2026-09-29 15:17:44 +0300 MSK
[doschennikov@archlinux ~]$ _
```

Рисунок 25. Состояние после остановки SIGKILL

7. После удаления:

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-nginx" --format
"table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
NAMES          STATUS          CREATED AT
[doschennikov@archlinux ~]$ _
```

Рисунок 26. Состояние после удаления

Шаг	Команда	STATUS
1	<code>docker create --name doschennikov-nginx nginx:alpine</code>	Created
2	<code>docker start doschennikov-nginx</code>	Up
3	<code>docker stop doschennikov-nginx</code>	Exited
4	<code>docker restart doschennikov-nginx</code>	Up
5	<code>docker kill doschennikov-nginx</code>	Exited
6	<code>docker rm doschennikov-nginx</code>	контейнер удалён

Табл. 2. Состояния

1. `docker stop` по умолчанию отправляет `SIGTERM` и через «grace period» (по умолчанию на linux - 10 сек, на windows - 30 сек; флаг `--timeout` (`-t`) позволяет установить свое значение) отправляется `SIGKILL`. Тем не менее первый сигнал можно поменять при помощи флага `--stop-signal` или инструкции `STOPSIGNAL` в Dockerfile контейнера.
 2. `docker kill` отправляет главному процессу `SIGKILL` (сигнал можно специфицировать флагом `--signal`), который вызывает немедленное завершение процесса. Если в момент отправления сигнала происходят взаимодействия с базой, то они могут быть не завершены корректно, что в свою очередь приведет к потере или повреждению данных.
- Если выполнить `docker rm` для запущенного контейнера без дополнительных параметров, Docker не удалит его и вернёт ошибку о том, что контейнер запущен. Для удаления запущенного контейнера принудительно используется флаг `-f` (`--force`). В этом случае Docker сначала принудительно завершает главный процесс контейнера, отправляя ему `SIGKILL`, после чего удаляет контейнер.

Этап 4. Диагностика реального сервиса

Задание 4.1

Я запустил контейнер с ошибкой:

```
docker run -d \  
  --name doschennikov-pg-broken \  
  -p 5527:5432 \  
  postgres:15
```

```
[doschennikov@archlinux ~]$ docker run -d \  
> --name doschennikov-pg-broken \  
> -p 5527:5432 \  
> postgres:15  
Unable to find image 'postgres:15' locally  
15: Pulling from library/postgres  
6b37362b3da7: Pull complete  
afe9a40fb214: Pull complete  
b65147f15943: Pull complete  
7b9c6b8e5e6e: Pull complete  
eb4d92aaee0a: Pull complete  
ca5e97c0cd7d: Pull complete  
c95afe7c234f: Pull complete  
53fa82bdba52: Pull complete  
bc176b99a33a: Pull complete  
0d791b2b75ae: Pull complete  
0d39881c83f7: Pull complete  
75a4dff090e8: Pull complete  
08a5cd487deb: Pull complete  
170d9ea1cd85: Pull complete  
5ecb84be746b: Download complete  
68ff3d2ea174: Download complete  
Digest: sha256:724292da1f2e50bdccfc3302ce75bbba7f4a6076701b588cc795fcac65683550  
Status: Downloaded newer image for postgres:15  
b47f45d8a312dd18477e2bb8e45410ae5084868fee1b209110980bf1d601bd84  
[doschennikov@archlinux ~]$ _
```

Рисунок 27. Запуск контейнера

Я проверил статус контейнера:

```
docker ps -a --filter "name=doschennikov-pg"
```

```
[doschennikov@archlinux ~]$ docker ps -a --filter "name=doschennikov-pg"  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS  
PORTS          NAMES  
b47f45d8a312   postgres:15 "docker-entrypoint.s..." 2 minutes ago  Exited (1) 2 minutes ago  
doschennikov-pg-broken
```

Рисунок 28. Статус контейнера

Затем я вывел логи с временной меткой:

```
docker logs --timestamps doschennikov-pg-broken 2>&1 | tail -20
```

```
[doschennikov@archlinux ~]$ docker logs --timestamps doschennikov-pg-broken 2>&1 | tail -20
2026-09-29T13:59:53.600026674Z Error: Database is uninitialized and superuser password is not specified.
2026-09-29T13:59:53.600060307Z         You must specify POSTGRES_PASSWORD to a non-empty value for the
2026-09-29T13:59:53.600072901Z         superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
2026-09-29T13:59:53.600076999Z
2026-09-29T13:59:53.600080636Z         You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
2026-09-29T13:59:53.600084282Z         connections without a password. This is *not* recommended.
2026-09-29T13:59:53.600087870Z
2026-09-29T13:59:53.600091437Z         See PostgreSQL documentation about "trust":
2026-09-29T13:59:53.600095314Z         https://www.postgresql.org/docs/current/auth-trust.html
[doschennikov@archlinux ~]$ _
```

Рисунок 29. Логи с временной меткой

Код завершения:

```
docker inspect doschennikov-pg-broken --format='{{.State.ExitCode}}'
```

```
[doschennikov@archlinux ~]$ docker inspect doschennikov-pg-broken --format='{{.State.ExitCode}}'
1
[doschennikov@archlinux ~]$ _
```

Рисунок 30. Код завершения

Задание 4.2

Исправил и проверил:

```
docker run -d \
  --name doschennikov-pg-fixed \
  -e POSTGRES_PASSWORD=Pass_5527 \
  -e POSTGRES_USER=user_465797 \
  -p 5527:5432 \
  postgres:15
```

```
[doschennikov@archlinux ~]$ docker run -d \
> --name doschennikov-pg-fixed \
> -e POSTGRES_PASSWORD=Pass_5527 \
> -e POSTGRES_USER=user_465797 \
> -p 5527:5432 \
> postgres:15
6b1e4c6490e9dcbee9e833502fdf9704443ec62b4215a49ddfb2673bc838b1a2
[doschennikov@archlinux ~]$ _
```

Рисунок 31. Исправленный вариант

Проверка работоспособности:

```
docker exec doschennikov-pg-fixed psql -U user_465797 -c "SELECT
version();"

```

```
[doschennikov@archlinux ~]$ docker exec doschennikov-pg-fixed psql -U user_465797 -c "SELECT version();"
          version
-----
 PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

[doschennikov@archlinux ~]$ _
```

Рисунок 32. Результат проверки

Задание 4.3

Финальный артефакт:

```
echo "=== Практика №1: $(date '+%Y-%m-%d %H:%M:%S') ===" && \
echo "Студент: $(whoami)@$(hostname)" && \
docker ps --filter "name=doschennikov-pg-fixed" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
[doschennikov@archlinux ~]$ echo "=== Практика №1: $(date '+%Y-%m-%d %H:%M:%S') ===" && \
> echo "Студент: $(whoami)@$(hostname)" && \
> docker ps --filter "name=doschennikov-pg-fixed" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
=== Практика №1: 2026-09-29 17:53:43 ===
Студент: doschennikov@archlinux
NAMES          STATUS          PORTS
doschennikov-pg-fixed Up 17 minutes  0.0.0.0:5527->5432/tcp, [::]:5527->5432/tcp
[doschennikov@archlinux ~]$ _
```

Рисунок 33. Финальный артефакт

Вывод

В ходе практической работы были изучены основные команды Docker, работа с образами и контейнерами, их жизненный цикл и диагностика ошибок. Были рассмотрены запуск, остановка, перезапуск и удаление контейнеров, а также работа с Docker Engine и PostgreSQL. Полученные навыки позволяют выполнять базовое управление контейнерами и диагностировать проблемы при их запуске.